Keras課堂實驗(四)

MNIST 數字辨識

高怡宣
陽明大學 生物醫學影像暨放射科學系

MNIST database

The MNIST database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits. The database is widely used for training and testing in the field of machine learning.

The MNIST database contains 60,000 training images and 10,000 testing images. The black and white images were fitted into a 28x28 pixel bounding box.

https://en.wikipedia.org/wiki/MNIST_database

By Josef Steppan - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=64810040

Data: Keras MNIST dataset
https://keras.io/datasets/#mnist-database-of-handwritten-digits

Code: TensorFlow+Keras[深度學習]人工智慧實務應用 / 林大貴

python

ANACONDA

Keras: The Python Deep Learning library

Keras

jupyter

TensorFlow

kaggle

1. MNIST MLP
2. Explanation on MLP
3. MNIST CNN – step 1
4. Explanation on CNN
5. MNIST CNN – step 2
6. Google Colab

jupyter Keras_Mnist_MLP Last Checkpoint: 幾秒前 (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Run Markdown

MNIST, Multi-Layer Perceptron (MLP)

MNIST由手寫阿拉伯數字組成，包含60,000個訓練樣本和10,000個測試樣本。

data from: https://keras.io/datasets/#mnist-database-of-handwritten-digits

code modified from: TensorFlow+Keras[深度學習]人工智慧實務應用 / 林大貴

(1) Import the data from Keras

In [1]:
```
from keras.utils import np_utils
from keras.models import Sequential
from keras.layers import Dense
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
np.random.seed(3)
```

```
C:\Users\USER\Anaconda3\lib\site-packages\h5py\__init__.py:36: FutureWar
argument of issubdtype from `float` to `np.floating` is deprecated. In f
np.float64 == np.dtype(float).type`.
  from .conv import register_converters as _register_converters
Using TensorFlow backend.
```

```
# Dataset of 60,000 28x28 grayscale images of the 10 digits, along with a test se

from keras.datasets import mnist
(x_train_image, y_train_label), (x_test_image, y_test_label) = mnist.load_data()

print(x_train_image.shape)
print(y_train_label.shape)
print(x_test_image.shape)
print(y_test_label.shape)
```

```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
```

Image size: 28 x 28

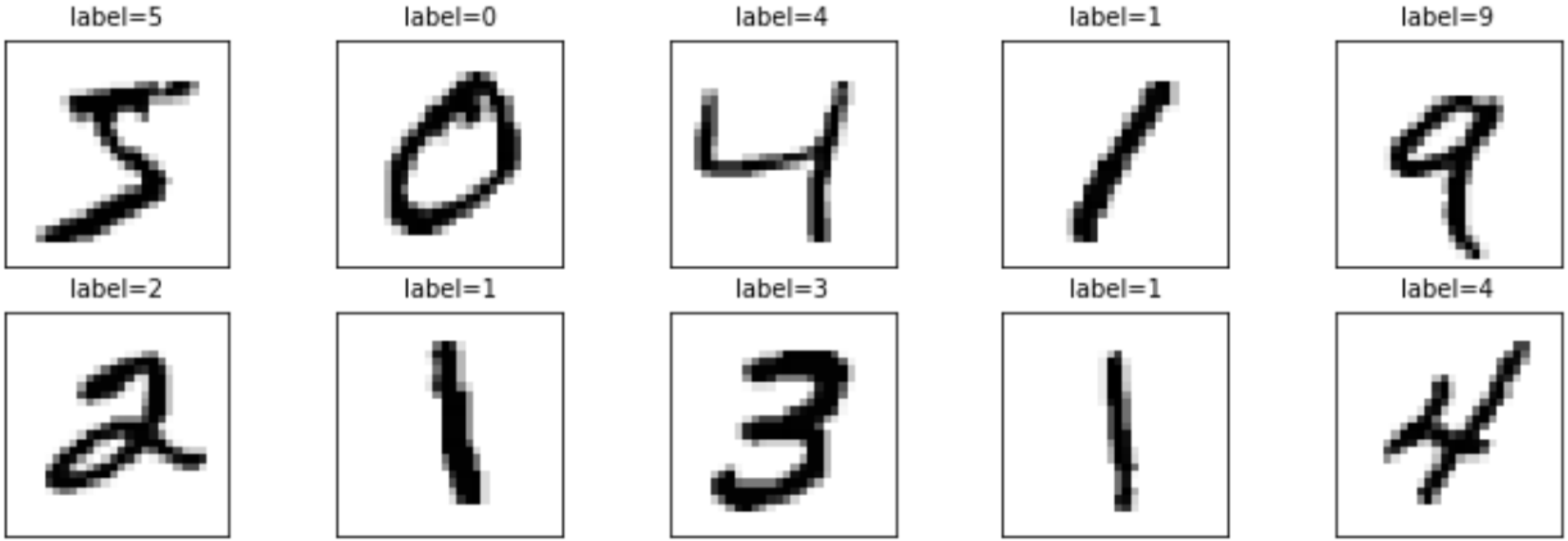
C:\Users\USER\.keras\datasets

名稱	修改日期	類型	大小
cifar-10-batches-py	2009/6/5 上午 04	檔案資料夾	
cifar-10-batches-py.tar.gz	2018/9/25 上午 1...	WinRAR 壓縮檔	166,503 KB
mnist.npz	2018/10/11 下午 ...	NPZ 檔案	11,222 KB

C:\Users\USER\.keras\datasets

(2) View the first 10 images and labels

In [3]:
```
fig = plt.gcf()
fig.set_size_inches(12,10)

for i in range(0,10):
    ax=plt.subplot(5,5,1+i)
    ax.imshow(x_train_image[i], cmap='binary')
    title= "label=" +str(y_train_label[i])
    ax.set_title(title,fontsize=10)
    ax.set_xticks([]);ax.set_yticks([])
plt.show()
```


label=5
label=0
label=4
label=1
label=9
label=2
label=1
label=3
label=1
label=4

(3) Convert 2-D image to 1-D array, normalize

In [4]:
```
# convert 2-D 28x28 image to 1-D 784 array

x_Train =x_train_image.reshape(60000, 784).astype('float32')
x_Test = x_test_image.reshape(10000, 784).astype('float32')
```

In [5]:
```
# normalize the image numbers to 0~1

x_Train_normalize = x_Train / 255
x_Test_normalize = x_Test / 255
print(x_Train_normalize.shape)
print(x_Test_normalize.shape)
```

```
(60000, 784)
(10000, 784)
```

(4) Convert label number to one-hot encoding

In [6]:
```
# convert label numbers to one-hot encoding
y_Train_OneHot = np_utils.to_categorical(y_train_label)
y_Test_OneHot = np_utils.to_categorical(y_test_label)
print(y_Train_OneHot.shape)
print(y_Test_OneHot.shape)
```

```
(60000, 10)
(10000, 10)
```

(5) Use a fully connected ANN

```
In [7]: model = Sequential()
```

```
In [8]: model.add(Dense(units=16,
                        input_dim=784,
                        kernel_initializer='normal',
                        activation='relu'))
```

```
In [9]: model.add(Dense(units=10,
                        kernel_initializer='normal',
                        activation='softmax'))
```

```
In [10]: print(model.summary())
```

In [10]:
```
print(model.summary())
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_1 (Dense)              (None, 16)                12560
_________________________________________________________________
dense_2 (Dense)              (None, 10)                170
=================================================================
Total params: 12,730
Trainable params: 12,730
Non-trainable params: 0
_________________________________________________________________
None
```

(784 + 1) x 16 = 12560

(16 + 1) x 10 = 170

(6) Model training

In [11]:
```
model.compile(loss='categorical_crossentropy',
              optimizer='adam', metrics=['accuracy'])
```

In [12]:
```
train_history =model.fit(x=x_Train_normalize,
                         y=y_Train_OneHot,validation_split=0.2,
                         epochs=20, batch_size=50,verbose=2)
```

```
Train on 48000 samples, validate on 12000 samples
Epoch 1/20
 - 1s - loss: 0.6110 - acc: 0.8388 - val_loss: 0.3044 - val_acc: 0.9152
Epoch 2/20
 - 1s - loss: 0.2961 - acc: 0.9156 - val_loss: 0.2641 - val_acc: 0.9247
Epoch 3/20
 - 1s - loss: 0.2611 - acc: 0.9254 - val_loss: 0.2367 - val_acc: 0.9347
```

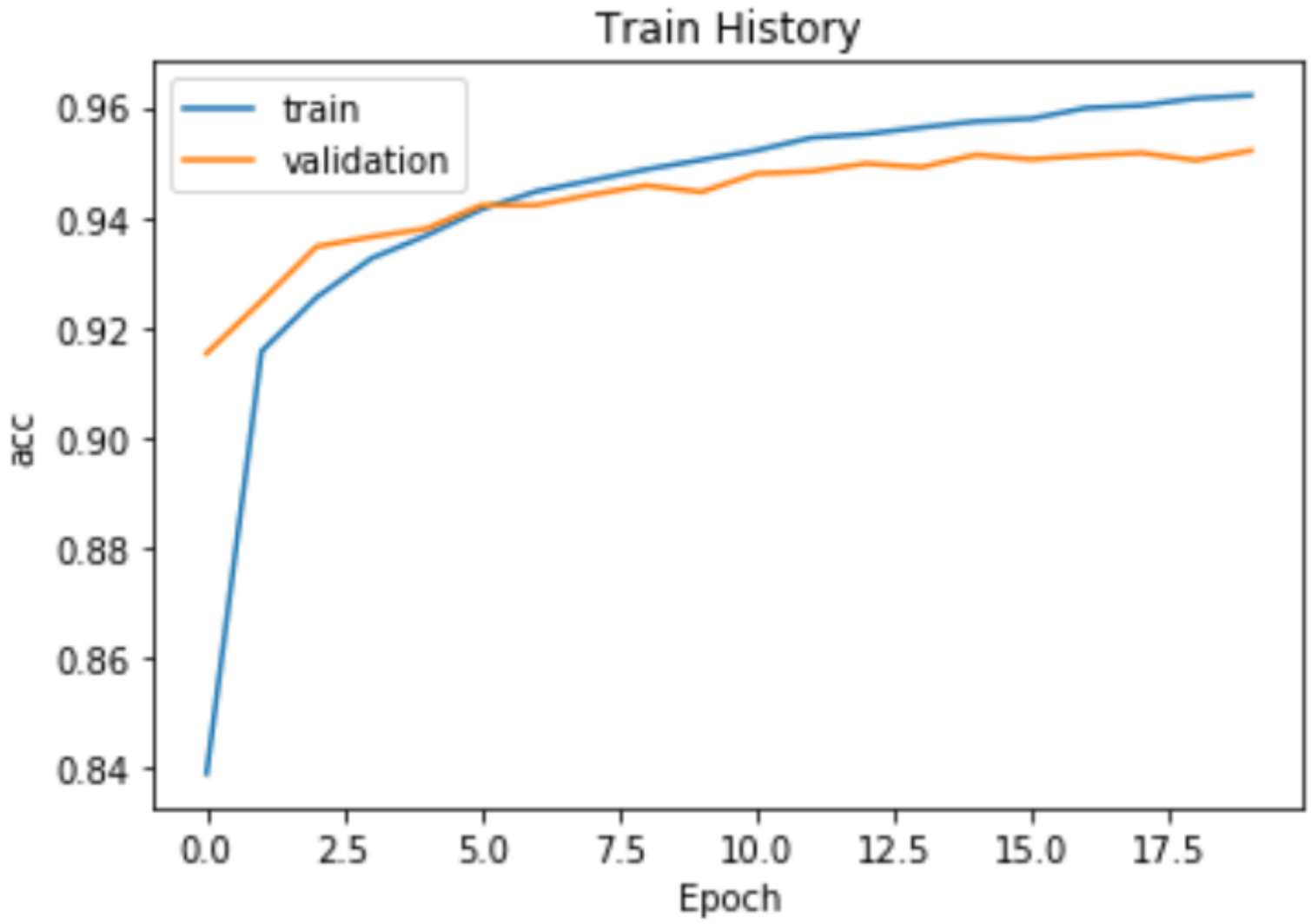
```
Epoch 13/20
 - 1s - loss: 0.1551 - acc: 0.9551 - val_loss: 0.1789 - val_acc: 0.9497
Epoch 14/20
 - 1s - loss: 0.1493 - acc: 0.9563 - val_loss: 0.1800 - val_acc: 0.9491
Epoch 15/20
 - 1s - loss: 0.1449 - acc: 0.9574 - val_loss: 0.1799 - val_acc: 0.9513
Epoch 16/20
 - 1s - loss: 0.1416 - acc: 0.9579 - val_loss: 0.1767 - val_acc: 0.9505
Epoch 17/20
 - 1s - loss: 0.1379 - acc: 0.9598 - val_loss: 0.1756 - val_acc: 0.9512
Epoch 18/20
 - 1s - loss: 0.1347 - acc: 0.9603 - val_loss: 0.1777 - val_acc: 0.9517
Epoch 19/20
 - 1s - loss: 0.1328 - acc: 0.9616 - val_loss: 0.1744 - val_acc: 0.9503
Epoch 20/20
 - 1s - loss: 0.1289 - acc: 0.9621 - val_loss: 0.1747 - val_acc: 0.9521
```

(7) Training history

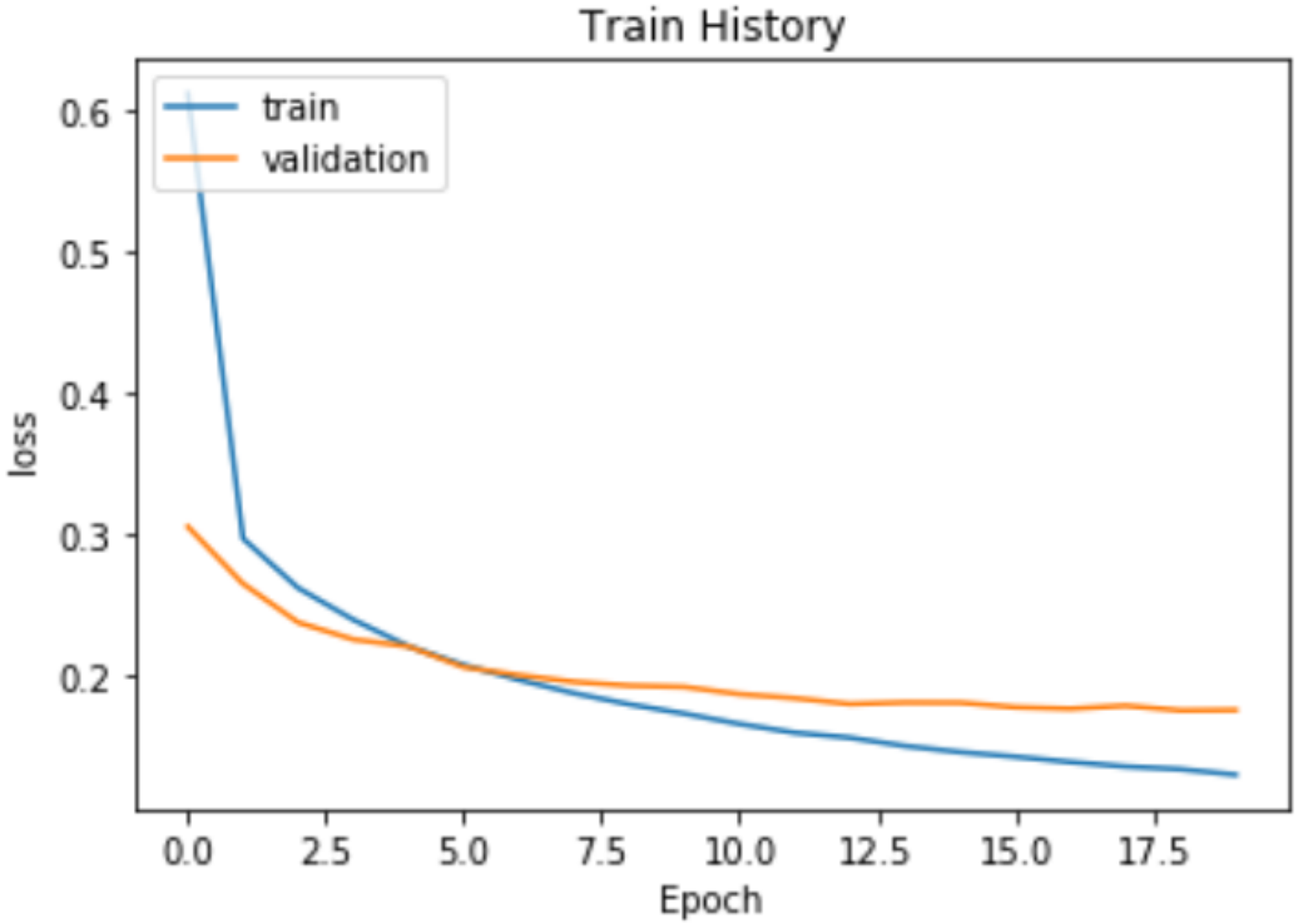
In [13]:
```
def show_train_history(train_history,train,validation):
    plt.plot(train_history.history[train])
    plt.plot(train_history.history[validation])
    plt.title('Train History')
    plt.ylabel(train)
    plt.xlabel('Epoch')
    plt.legend(['train', 'validation'], loc='upper left')
    plt.show()
```

In [14]:
```
show_train_history(train_history,'acc','val_acc')
```

```
In [14]: show_train_history(train_history,'acc','val_acc')
```



```
In [15]: show_train_history(train_history,'loss','val_loss')
```



(8) Accuracy

In [16]:
```
scores = model.evaluate(x_Test_normalize, y_Test_OneHot)
print()
print('accuracy=',scores[1])
```

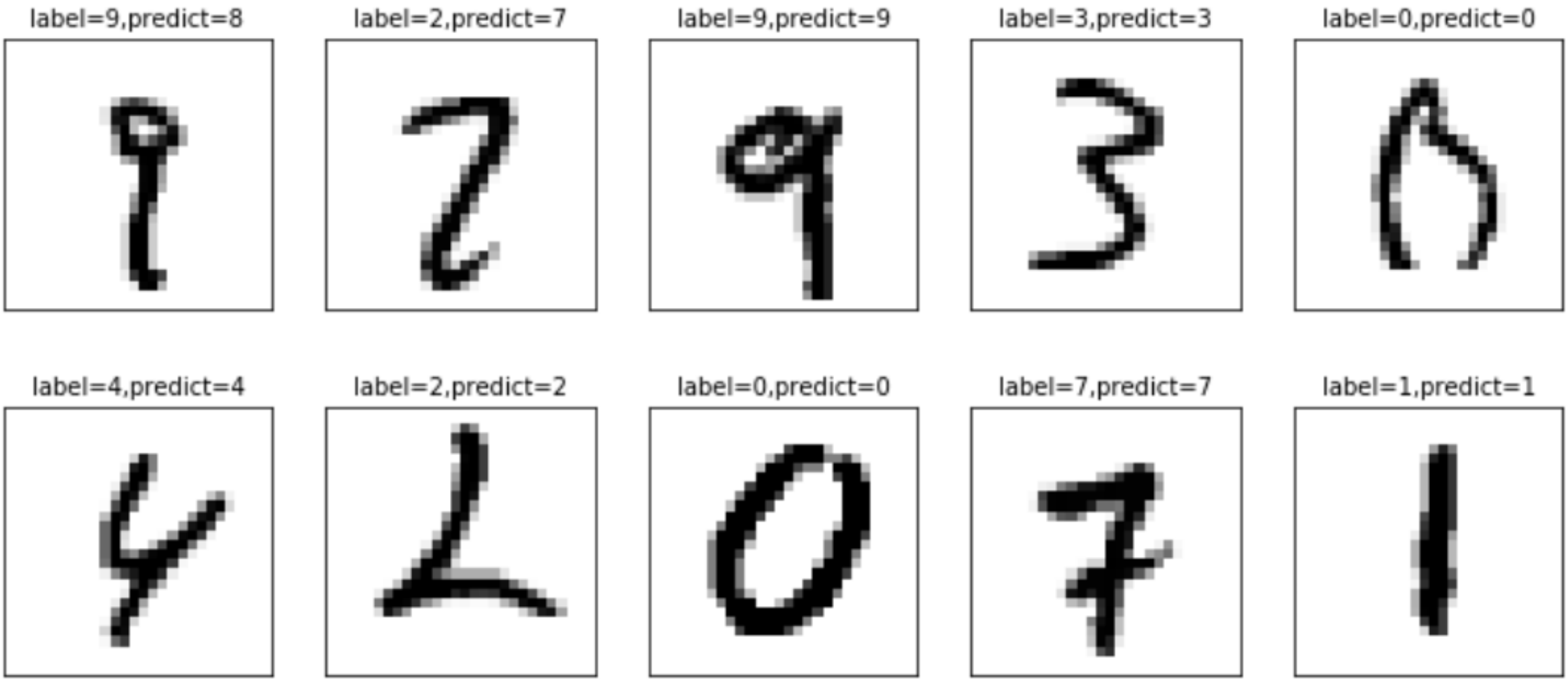
```
10000/10000 [==============================] - 1s 53us/step

accuracy= 0.9527
```

(9) Prediction

In [17]:
```
prediction=model.predict_classes(x_Test_normalize)
```

In [18]:
```
def plot_images_labels_prediction(images,labels,prediction,
                                  idx,num=10):
    fig = plt.gcf()
    fig.set_size_inches(12, 10)
    if num>25: num=25
    for i in range(0, num):
        ax=plt.subplot(5,5, 1+i)
        ax.imshow(images[idx], cmap='binary')
        title= "label=" +str(labels[idx])
        if len(prediction)>0:
            title+=",predict="+str(prediction[idx])

        ax.set_title(title,fontsize=10)
        ax.set_xticks([]);ax.set_yticks([])
        idx+=1
    plt.show()
```

```
plot_images_labels_prediction(x_test_image,y_test_label,
                              prediction,idx=320)
```



(10) Confusion matrix


```
In [24]: pd.crosstab(y_test_label,prediction,
                     rownames=['label'],colnames=['predict'])
```

predict	0	1	2	3	4	5	6	7	8	9
label										
0	963	0	2	0	1	4	6	2	2	0
1	0	1116	3	0	0	1	4	2	9	0
2	8	5	981	5	4	1	7	11	9	1
3	4	1	9	944	2	24	0	7	15	4
4	1	1	3	2	945	0	7	2	2	19
5	6	3	2	20	2	829	12	2	12	4
6	6	2	4	0	7	8	928	0	3	0
7	0	6	20	10	3	1	0	970	1	17
8	6	3	5	13	13	9	7	7	902	9
9	5	5	2	11	17	3	0	9	8	949

1. MNIST MLP
2. Explanation on MLP
3. MNIST CNN – step 1
4. Explanation on CNN
5. MNIST CNN – step 2
6. Google Colab

2. Explanation on MLP

究竟神經網路是什麼？ | 第一章 深度學習
But what *is* a Neural Network?

https://www.youtube.com/watch?v=aircAruvnKk

1. MNIST MLP
2. Explanation on MLP
3. MNIST CNN – step 1
4. Explanation on CNN
5. MNIST CNN – step 2
6. Google Colab

jupyter Keras_Mnist_CNN_step_1 (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Run Markdown

MNIST, Convolutional Neural Network (CNN)

Step - 1

MNIST由手寫阿拉伯數字組成，包含60,000個訓練樣本和10,000個測試樣本。

data from: https://keras.io/datasets/#mnist-database-of-handwritten-digits

code modified from: TensorFlow+Keras[深度學習]人工智慧實務應用 / 林大貴

(3) Convert 2-D image to nx28x28x1 array, normalize the numbers

In [4]:
```
# convert 2-D 28x28 image to 1-D 784 array
# x_Train =x_train_image.reshape(60000, 784).astype('float32')
# x_Test = x_test_image.reshape(10000, 784).astype('float32')

# convert 2-D 28x28 image to 4-D nx28x28x1  array
x_Train4D=x_train_image.reshape(x_train_image.shape[0],28,28,1).astype('float32')
x_Test4D=x_test_image.reshape(x_test_image.shape[0],28,28,1).astype('float32')
```

In [5]:
```
# normalize the image numbers to 0~1

x_Train4D_normalize = x_Train4D / 255
x_Test4D_normalize = x_Test4D / 255
print(x_Train4D_normalize.shape)
print(x_Test4D_normalize.shape)
```

```
(60000, 28, 28, 1)
(10000, 28, 28, 1)
```

(4) Convert label number to one-hot encoding

```
# convert label numbers to one-hot encoding
y_TrainOneHot = np_utils.to_categorical(y_train_label)
y_TestOneHot = np_utils.to_categorical(y_test_label)
print(y_TrainOneHot.shape)
print(y_TestOneHot.shape)
```

```
(60000, 10)
(10000, 10)
```

(5) Use a Convolutional Neural Network

```
from keras.models import Sequential
from keras.layers import Dense,Dropout,Flatten,Conv2D,MaxPooling2D
```

```
model = Sequential()
```

```
model.add(Conv2D(filters=16,
                 kernel_size=(5,5),
                 padding='same',
                 input_shape=(28,28,1),
                 activation='relu'))
```

```
model.add(Flatten())
```

```
# Enable this cell in the second step

# model.add(Dense(128, activation='relu'))
# model.add(Dropout(0.5))
```

```
model.add(Dense(10,activation='softmax'))
```

```
print(model.summary())
```

```python
print(model.summary())
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 28, 28, 16)        416
_________________________________________________________________
flatten_1 (Flatten)          (None, 12544)             0
_________________________________________________________________
dense_1 (Dense)              (None, 10)                125450
=================================================================
Total params: 125,866
Trainable params: 125,866
Non-trainable params: 0
_________________________________________________________________
None
```

(5x5 + 1) x 16 = 416

28 x 28 x 16 = 12544

(12544 + 1) x 10 = 125450

(6) Model training

```
model.compile(loss='categorical_crossentropy',
              optimizer='adam',metrics=['accuracy'])
```

```
train_history=model.fit(x=x_Train4D_normalize,
                        y=y_TrainOneHot,validation_split=0.2,
                        epochs=10, batch_size=300,verbose=2)
```

```
Train on 48000 samples, validate on 12000 samples
Epoch 1/10
 - 18s - loss: 0.3954 - acc: 0.8891 - val_loss: 0.1923 - val_acc: 0.9467
Epoch 2/10
 - 17s - loss: 0.1553 - acc: 0.9552 - val_loss: 0.1234 - val_acc: 0.9669
Epoch 3/10
 - 17s - loss: 0.1051 - acc: 0.9700 - val_loss: 0.1000 - val_acc: 0.9735
```

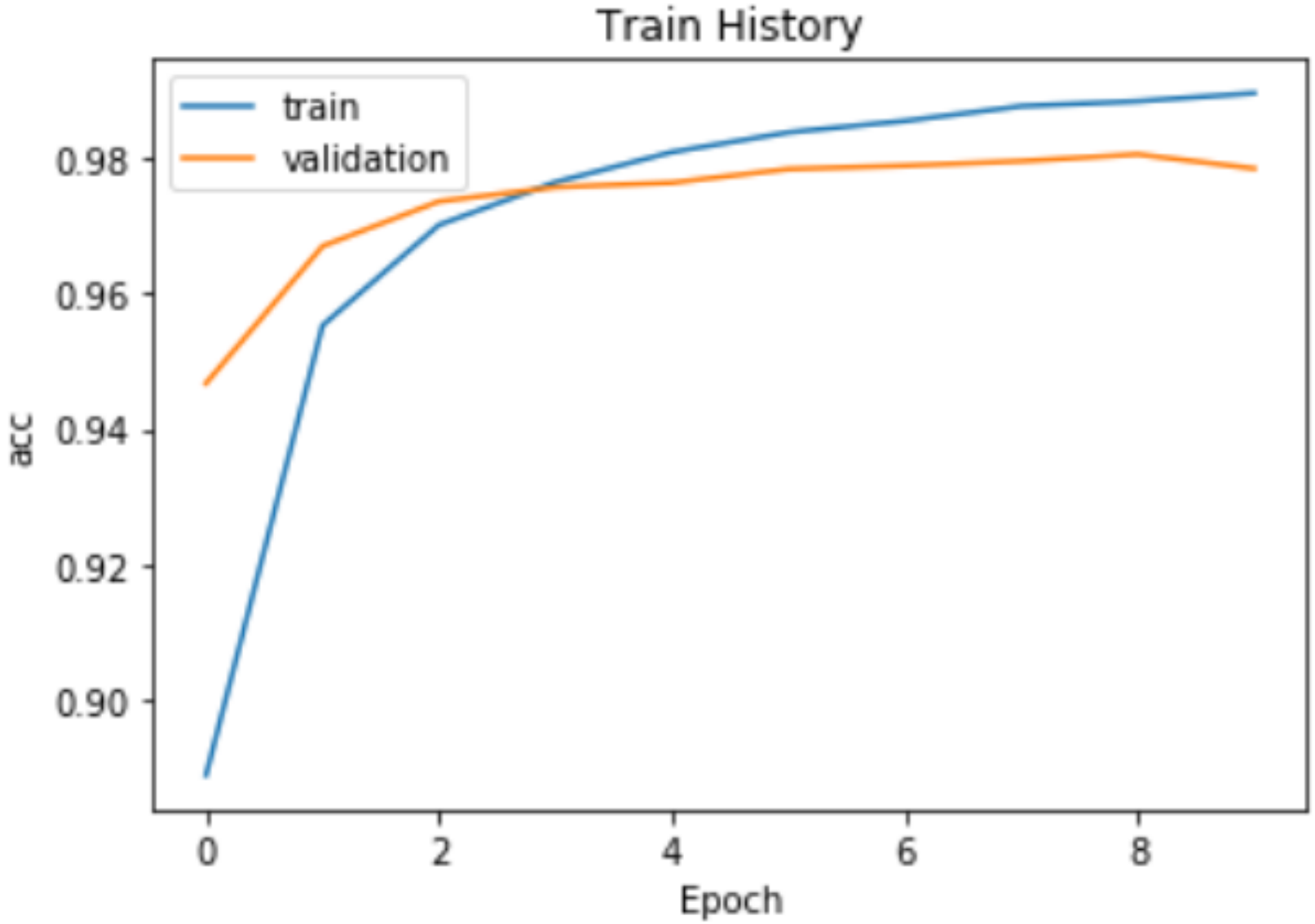
```
Epoch 4/10
 - 18s - loss: 0.0823 - acc: 0.9763 - val_loss: 0.0848 - val_acc: 0.9756
Epoch 5/10
 - 18s - loss: 0.0683 - acc: 0.9808 - val_loss: 0.0796 - val_acc: 0.9763
Epoch 6/10
 - 17s - loss: 0.0590 - acc: 0.9836 - val_loss: 0.0750 - val_acc: 0.9783
Epoch 7/10
 - 17s - loss: 0.0514 - acc: 0.9853 - val_loss: 0.0722 - val_acc: 0.9788
Epoch 8/10
 - 18s - loss: 0.0443 - acc: 0.9875 - val_loss: 0.0723 - val_acc: 0.9794
Epoch 9/10
 - 17s - loss: 0.0407 - acc: 0.9882 - val_loss: 0.0700 - val_acc: 0.9804
Epoch 10/10
 - 18s - loss: 0.0359 - acc: 0.9894 - val_loss: 0.0710 - val_acc: 0.9783
```

(7) Training history

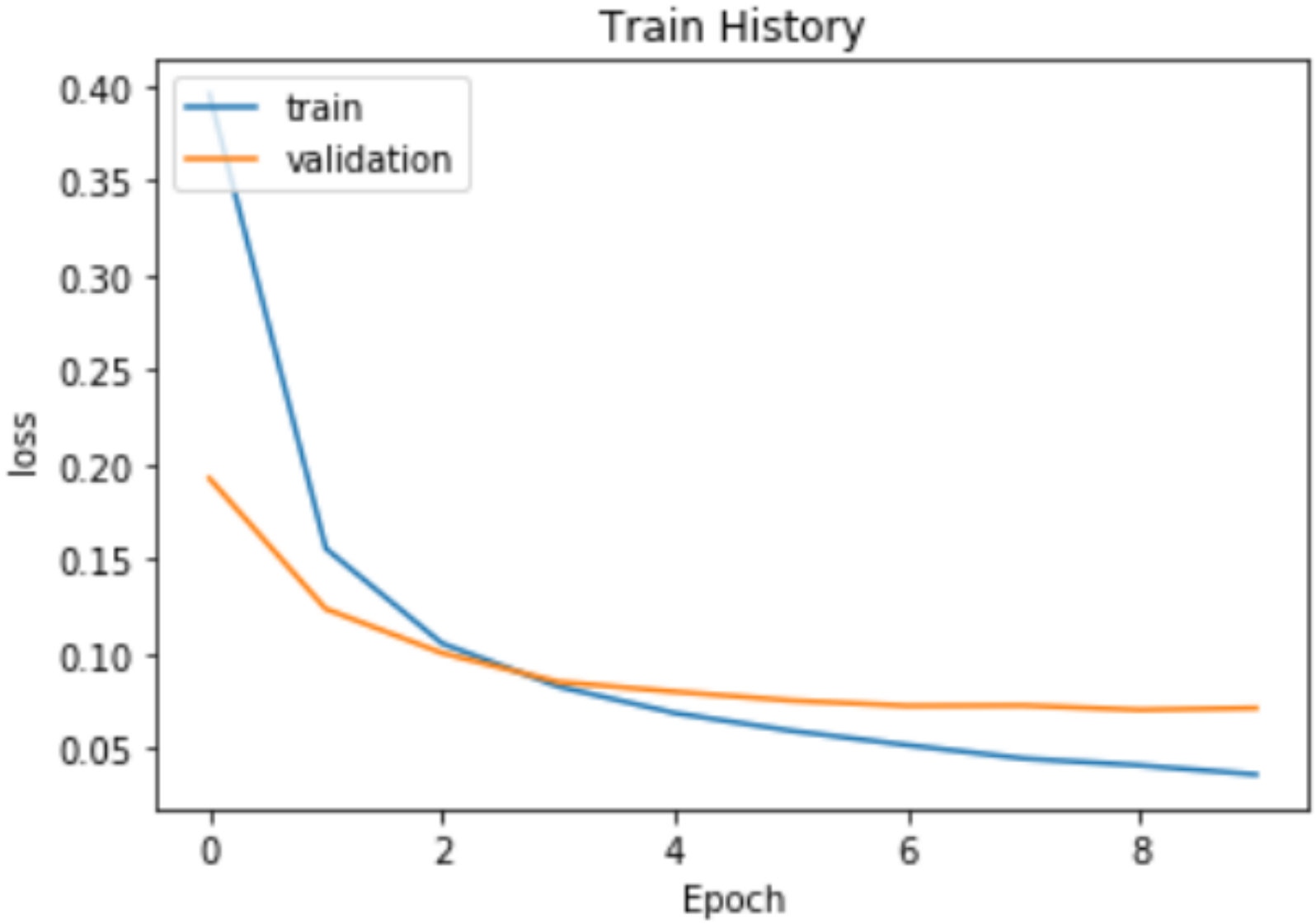
In [17]:
```
def show_train_history(train_history,train,validation):
    plt.plot(train_history.history[train])
    plt.plot(train_history.history[validation])
    plt.title('Train History')
    plt.ylabel(train)
    plt.xlabel('Epoch')
    plt.legend(['train', 'validation'], loc='upper left')
    plt.show()
```

In [18]:
```
show_train_history(train_history,'acc','val_acc')
```

```
In [18]: show_train_history(train_history,'acc','val_acc')
```



```
In [19]: show_train_history(train_history,'loss','val_loss')
```



(8) Accuracy

In [20]:
```
scores = model.evaluate(x_Test4D_normalize, y_TestOneHot)
print()
print('accuracy=',scores[1])
```

```
10000/10000 [==============================] - 5s 512us/step

accuracy= 0.9821
```

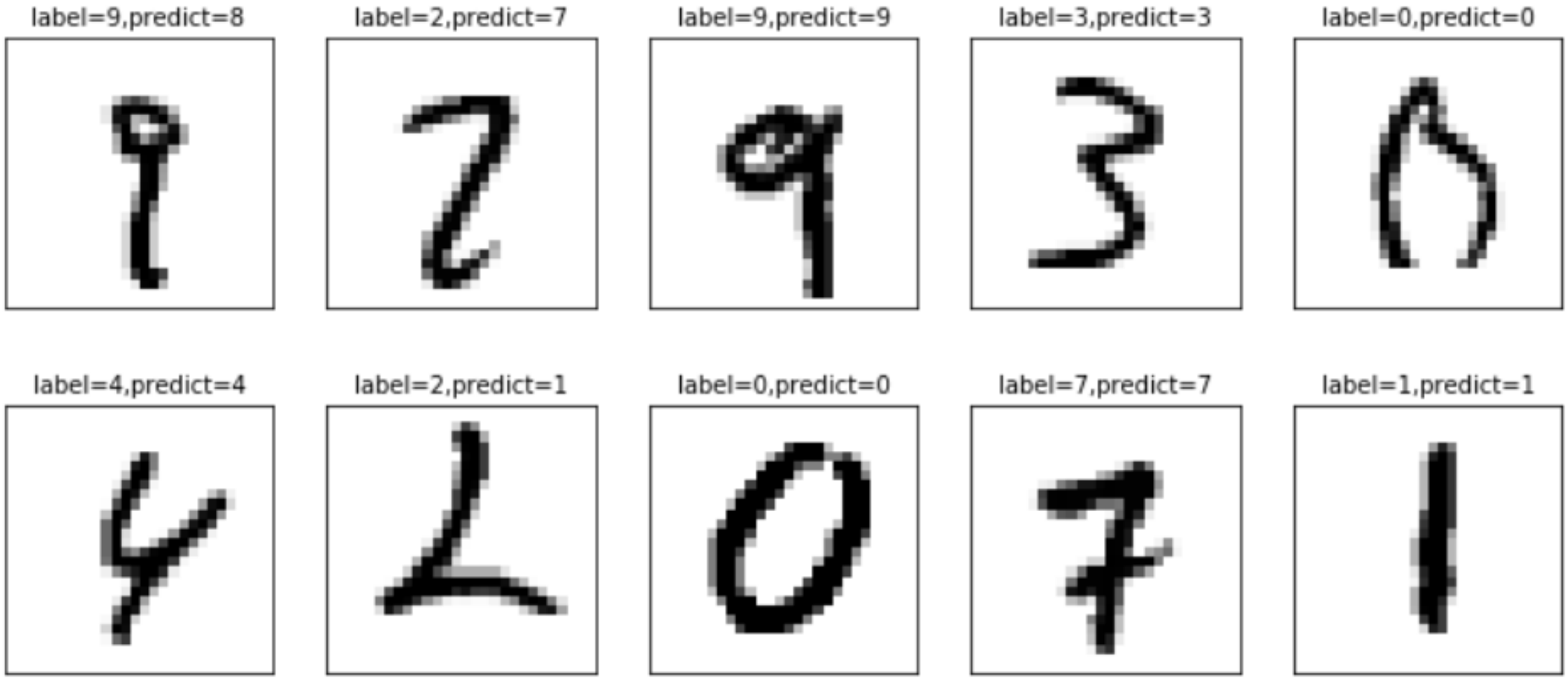
(9) Prediction

In [21]:
```
prediction=model.predict_classes(x_Test4D_normalize)
```

In [21]:
```
prediction=model.predict_classes(x_Test4D)
```

In [22]:
```
def plot_images_labels_prediction(images,labels,prediction,
                                  idx,num=10):
    fig = plt.gcf()
    fig.set_size_inches(12, 14)
    if num>25: num=25
    for i in range(0, num):
        ax=plt.subplot(5,5, 1+i)
        ax.imshow(images[idx], cmap='binary')
        title= "label=" +str(labels[idx])
        if len(prediction)>0:
            title+=",predict="+str(prediction[idx])

        ax.set_title(title,fontsize=10)
        ax.set_xticks([]);ax.set_yticks([])
        idx+=1
    plt.show()
```

```
plot_images_labels_prediction(x_test_image,y_test_label,
                              prediction,idx=320)
```



(10) Confusion matrix


```
In [24]: pd.crosstab(y_test_label,prediction,
                     rownames=['label'],colnames=['predict'])
```

predict	0	1	2	3	4	5	6	7	8	9
label										
0	974	0	0	1	0	1	1	1	2	0
1	0	1129	1	2	0	0	3	0	0	0
2	2	5	1002	1	1	0	1	10	9	1
3	1	0	3	991	0	11	0	1	2	1
4	1	1	1	1	971	0	1	0	1	5
5	3	0	0	4	0	878	4	2	1	0
6	7	2	0	0	2	2	942	0	3	0
7	1	4	5	4	1	0	0	1008	3	2
8	6	0	2	1	0	1	2	4	954	4
9	3	5	0	2	7	4	0	11	5	972

1. MNIST MLP
2. Explanation on MLP
3. MNIST CNN – step 1
4. Explanation on CNN
5. MNIST CNN – step 2
6. Google Colab

4. Explanation on CNN

什么是卷积神经网络 CNN (深度学习)?
https://www.youtube.com/watch?v=hMIZ85t9r9A

机器学习-什么是卷积网络
https://www.youtube.com/watch?v=QsxKKyhYxFQ

1. MNIST MLP
2. Explanation on MLP
3. MNIST CNN – step 1
4. Explanation on CNN
5. MNIST CNN – step 2
6. Google Colab

jupyter Keras_Mnist_CNN_step_2 Last Checkpoint: 幾秒前 (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Run Code

MNIST, Convolutional Neural Network (CNN)

Step - 2

MNIST由手寫阿拉伯數字組成，包含60,000個訓練樣本和10,000個測試樣本。

data from: https://keras.io/datasets/#mnist-database-of-handwritten-digits

code modified from: TensorFlow+Keras[深度學習]人工智慧實務應用 / 林大貴

(5) Use a Convolutional Neural Network

```
from keras.models import Sequential
from keras.layers import Dense,Dropout,Flatten,Conv2D,MaxPooling2D
```

```
model = Sequential()
```

```
model.add(Conv2D(filters=16,
                 kernel_size=(5,5),
                 padding='same',
                 input_shape=(28,28,1),
                 activation='relu'))
```

In [10]:
```
# Enable this cell in the second step

model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(filters=36,
                 kernel_size=(5,5),
                 padding='same',
                 activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))
```

In [11]:
```
model.add(Flatten())
```

In [12]:
```
# Enable this cell in the second step

model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
```

In [13]:
```
model.add(Dense(10,activation='softmax'))
```

In [14]:
```
print(model.summary())
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 28, 28, 16)        416
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 14, 14, 16)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 14, 14, 36)        14436
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 7, 7, 36)          0
_________________________________________________________________
dropout_1 (Dropout)          (None, 7, 7, 36)          0
_________________________________________________________________
flatten_1 (Flatten)          (None, 1764)              0
_________________________________________________________________
dense_1 (Dense)              (None, 128)               225920
_________________________________________________________________
dropout_2 (Dropout)          (None, 128)               0
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1290
=================================================================
Total params: 242,062
```

(6) Model training

In [15]:
```
model.compile(loss='categorical_crossentropy',
              optimizer='adam',metrics=['accuracy'])
```

In [16]:
```
train_history=model.fit(x=x_Train4D_normalize,
                        y=y_TrainOneHot,validation_split=0.2,
                        epochs=50, batch_size=300,verbose=2)
```

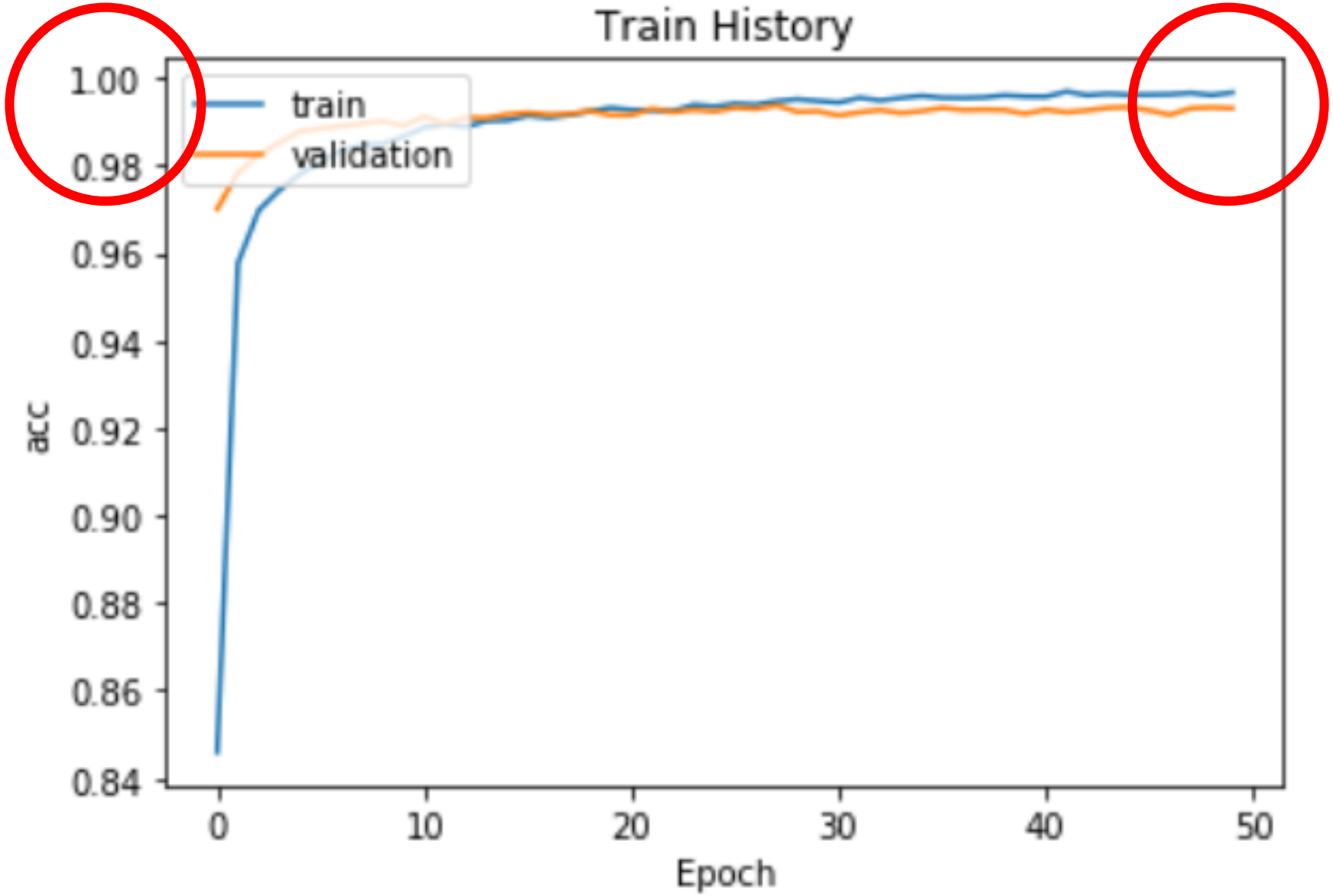
```
Train on 48000 samples, validate on 12000 samples
Epoch 1/50
 - 52s - loss: 0.4859 - acc: 0.8459 - val_loss: 0.1035 - val_acc: 0.9701
Epoch 2/50
 - 49s - loss: 0.1426 - acc: 0.9577 - val_loss: 0.0661 - val_acc: 0.9783
Epoch 3/50
 - 49s - loss: 0.1031 - acc: 0.9697 - val_loss: 0.0568 - val_acc: 0.9824
Epoch 4/50
```

```
Epoch 43/50
 - 47s - loss: 0.0114 - acc: 0.9961 - val_loss: 0.0335 - val_acc: 0.9926
Epoch 44/50
 - 47s - loss: 0.0100 - acc: 0.9964 - val_loss: 0.0314 - val_acc: 0.9932
Epoch 45/50
 - 47s - loss: 0.0113 - acc: 0.9962 - val_loss: 0.0304 - val_acc: 0.9933
Epoch 46/50
 - 47s - loss: 0.0110 - acc: 0.9962 - val_loss: 0.0319 - val_acc: 0.9926
Epoch 47/50
 - 47s - loss: 0.0104 - acc: 0.9963 - val_loss: 0.0375 - val_acc: 0.9916
Epoch 48/50
 - 47s - loss: 0.0100 - acc: 0.9966 - val_loss: 0.0321 - val_acc: 0.9931
Epoch 49/50
 - 47s - loss: 0.0114 - acc: 0.9961 - val_loss: 0.0332 - val_acc: 0.9933
Epoch 50/50
 - 47s - loss: 0.0105 - acc: 0.9967 - val_loss: 0.0307 - val_acc: 0.9931
```

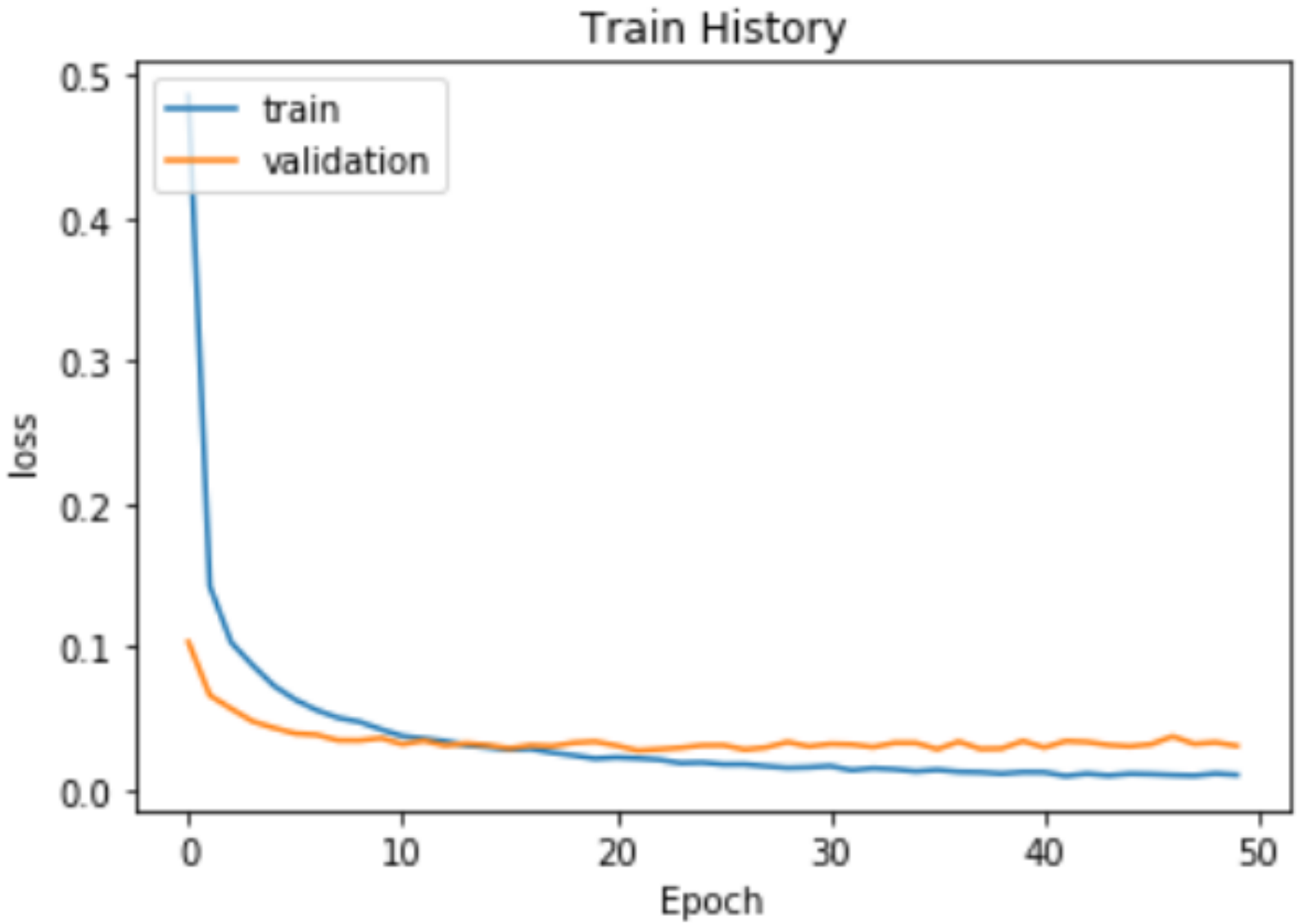
(7) Training history

```
def show_train_history(train_history,train,validation):
    plt.plot(train_history.history[train])
    plt.plot(train_history.history[validation])
    plt.title('Train History')
    plt.ylabel(train)
    plt.xlabel('Epoch')
    plt.legend(['train', 'validation'], loc='upper left')
    plt.show()
```

```
In [18]: show_train_history(train_history,'acc','val_acc')
```



```
In [19]: show_train_history(train_history,'loss','val_loss')
```



(8) Accuracy

In [20]:
```
scores = model.evaluate(x_Test4D_normalize, y_TestOneHot)
print()
print('accuracy=',scores[1])
```

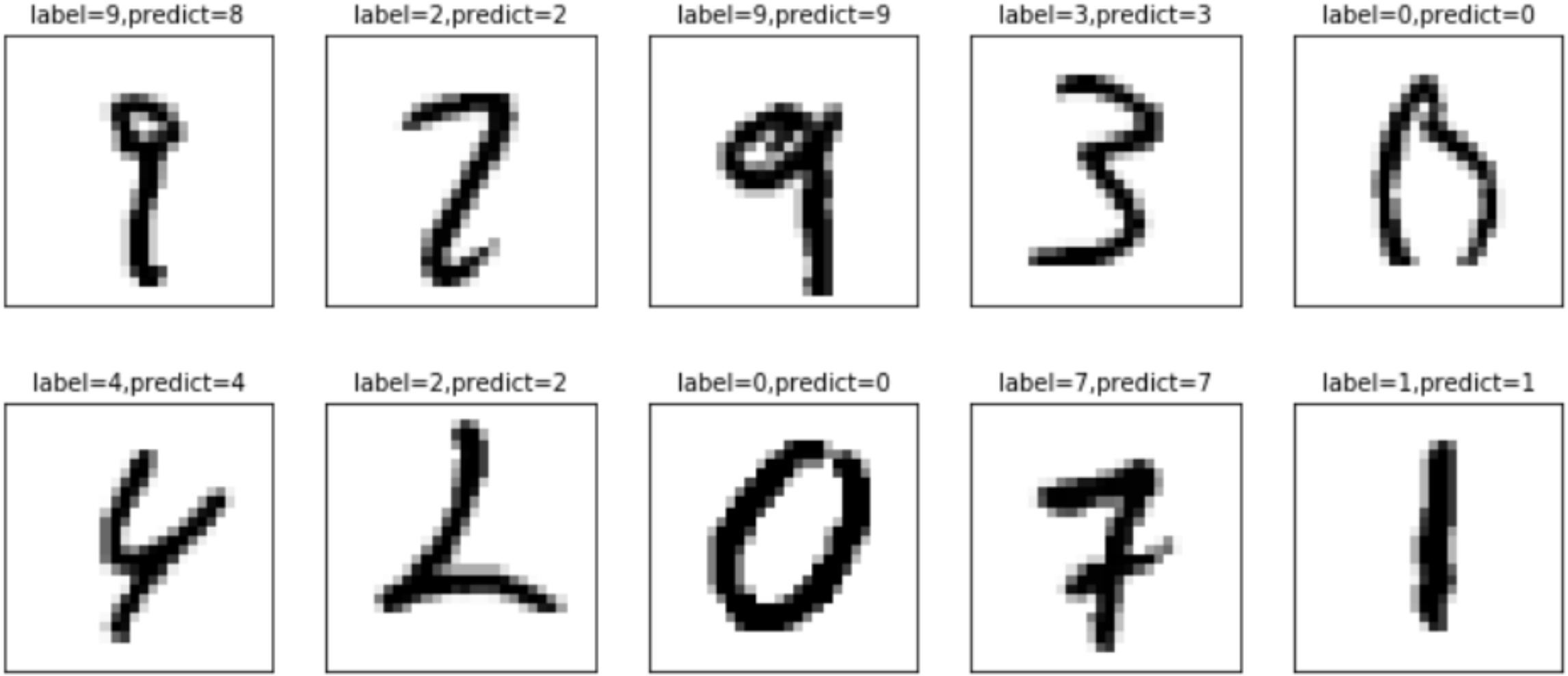
```
10000/10000 [==============================] - 4s 417us/step

accuracy= 0.9937
```

(9) Prediction

In [21]:
```
prediction=model.predict_classes(x_Test4D_normalize)
```

In [22]:
```
def plot_images_labels_prediction(images,labels,prediction,
                                  idx,num=10):
    fig = plt.gcf()
    fig.set_size_inches(12, 14)
    if num>25: num=25
    for i in range(0, num):
        ax=plt.subplot(5,5, 1+i)
        ax.imshow(images[idx], cmap='binary')
        title= "label=" +str(labels[idx])
        if len(prediction)>0:
            title+=",predict="+str(prediction[idx])

        ax.set_title(title,fontsize=10)
        ax.set_xticks([]);ax.set_yticks([])
        idx+=1
    plt.show()
```

```
plot_images_labels_prediction(x_test_image,y_test_label,
                              prediction,idx=320)
```



(10) Confusion matrix

In [24]:
```
pd.crosstab(y_test_label,prediction,
            rownames=['label'],colnames=['predict'])
```

predict	0	1	2	3	4	5	6	7	8	9
label										
0	978	0	0	0	0	0	0	1	1	0
1	0	1133	1	0	0	0	0	0	1	0
2	0	0	1028	0	0	0	0	4	0	0
3	0	0	3	1003	0	2	0	1	1	0
4	0	0	0	0	976	0	0	1	1	4
5	1	0	0	4	0	886	1	0	0	0
6	3	1	0	1	1	8	943	0	1	0
7	0	2	2	1	0	0	0	1022	1	0
8	2	0	1	1	0	0	0	1	968	1
9	0	2	0	1	5	3	0	3	2	993

1. MNIST MLP
2. Explanation on MLP
3. MNIST CNN – step 1
4. Explanation on CNN
5. MNIST CNN – step 2
6. Google Colab

雲端硬碟
搜尋雲端硬碟
新增
我的雲端硬碟
電腦
與我共用
近期存取
已加星號
垃圾桶
我的雲端硬碟 > MNIST
名稱
擁有者
上次修改日期
Keras_Mnist_CNN_step_3.ipynb
我
上午9:44
Double click this file
成功上傳 1 個項目
Keras_Mnist_CNN_step_3.ipynb

Keras_Mnist_CNN_step_3.ipynb
使用「Colaboratory」開啟
我的雲端硬碟 > MNIST
新增
名稱
擁有者
我的雲端硬碟
電腦
與我共用
近期存取
已加星號
垃圾桶
無法預覽
下載
連結更多應用程式...
建議使用下列其中一個應用程式開啟或編輯這個項目
已連結的應用程式
Colaboratory

Keras_Mnist_CNN_step_3.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

评论 共享

代码 文字 单元格 单元格 连接 修改

How to use Google Colab

MNIST, Convolutional Neural Network (CNN)

Step - 3

MNIST由手寫阿拉伯數字組成，包含60,000個訓練樣本和10,000個測試樣本。

data from: https://keras.io/datasets/#mnist-database-of-handwritten-digits

code modified from: TensorFlow+Keras[深度學習]人工智慧實務應用 / 林大貴

Keras_Mnist_CNN_step_3.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

代码 文字 单

全部运行	Ctrl+F9
运行当前单元格之前的所有单元格	Ctrl+F8
运行光标所在的单元格	Ctrl+Enter
运行所选单元格的内容	Ctrl+Shift+Enter
运行当前单元格以及其后的所有单元格	Ctrl+F10
中断执行	Ctrl+M I
重新启动代码执行程序...	Ctrl+M .
重新启动并运行所有单元格...	
重置所有代码执行程序...	
更改运行时类型	
管理会话	

How to use Go

MNIST, Convolutional N

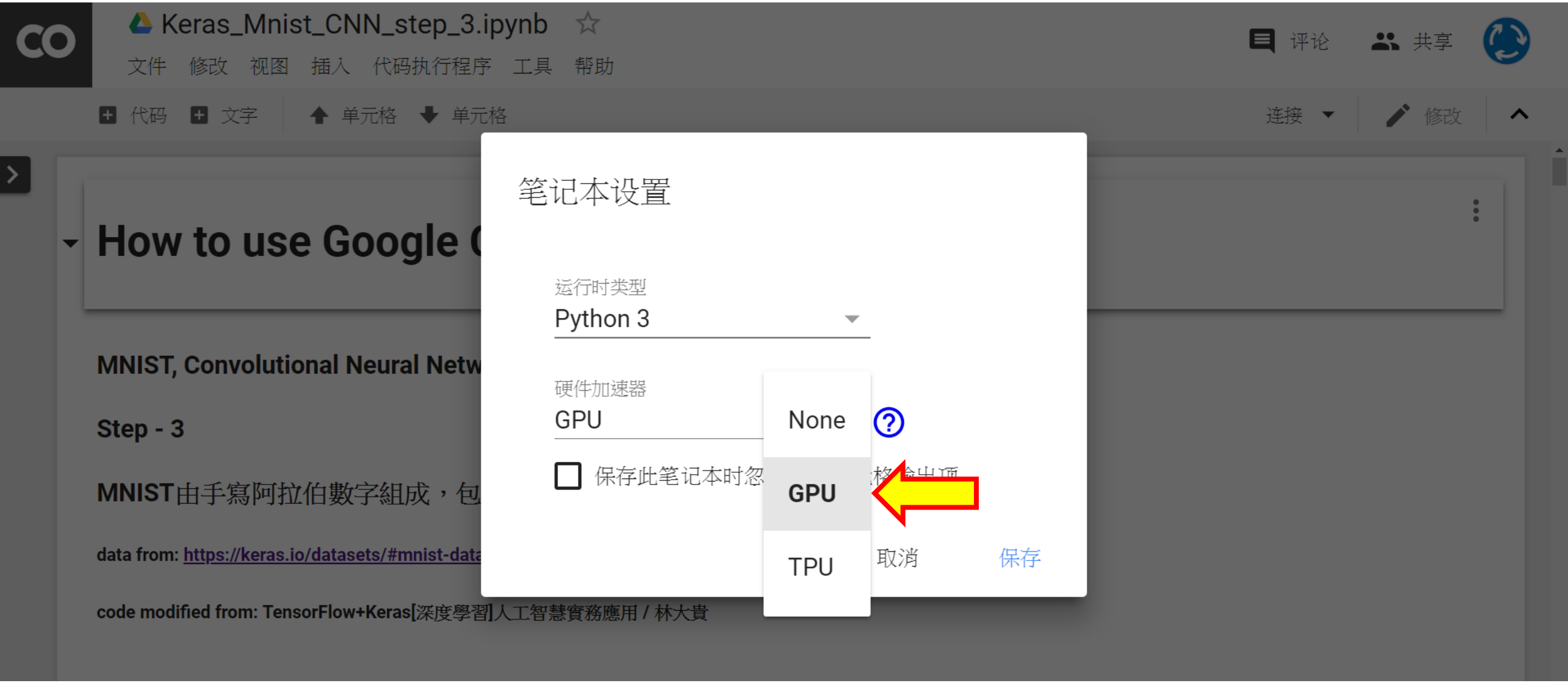
Step - 3

MNIST由手寫阿拉伯數字

本。

data from: https://keras.io/datase

code modified from: TensorFlow+


Keras_Mnist_CNN_step_3.ipynb
文件 修改 视图 插入 代码执行程序 工具 帮助
评论
共享
代码 文字 单元格 单元格
连接
修改
How to use Google C
MNIST, Convolutional Netw
Step - 3
MNIST由手寫阿拉伯數字組成，包
data from: https://keras.io/datasets/#mnist-data
code modified from: TensorFlow+Keras[深度學習]人工智慧實務應用 / 林大貴
笔记本设置
运行时类型
Python 3
硬件加速器
GPU
None
GPU
TPU
保存此笔记本时忽
取消
保存

Keras_Mnist_CNN_step_3.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

代码 文字 单

全部运行	Ctrl+F9
运行当前单元格之前的所有单元格	Ctrl+F8
运行光标所在的单元格	Ctrl+Enter
运行所选单元格的内容	Ctrl+Shift+Enter
运行当前单元格以及其后的所有单元格	Ctrl+F10
中断执行	Ctrl+M I
重新启动代码执行程序...	Ctrl+M .
重新启动并运行所有单元格...	
重置所有代码执行程序...	
更改运行时类型	
管理会话	

How to use Go

MNIST, Convolutional N

Step - 3

MNIST由手寫阿拉伯數字

本。

data from: https://keras.io/datase

code modified from: TensorFlow+

Keras_Mnist_CNN_step_3.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

代码 文字 单元格 单元格

(1) Import the data from Keras

```
from keras.utils import np_utils
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
np.random.seed(3)
```

```
Using TensorFlow backend.
```

Keras_Mnist_CNN_step_3.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

代码 文字 单元格 单元格 忙碌

[25]
```
from keras.utils import np_utils
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
np.random.seed(3)
```

```
# (x_train_image, y_train_label), (x_test_image, y_test_label) = mnist.load_data()
# upload a file to Google CoLab

from google.colab import files
uploaded = files.upload() # then browse, select the files. It's then uploaded
```

選擇檔案 未選擇任何檔案 Cancel upload

C:\Users\USER\.keras\datasets

名稱	修改日期	類型	大小
cifar-10-batches-py	2009/6/5 上午 04	檔案資料夾	
cifar-10-batches-py.tar.gz	2018/9/25 上午 1...	WinRAR 壓縮檔	166,503 KB
mnist.npz	2018/10/11 下午 ...	NPZ 檔案	11,222 KB

C:\Users\USER\.keras\datasets

Keras_Mnist_CNN_step_3.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

评论 共享

代码 文字 单元格

[25]
```
from keras.utils import
import matplotlib.pyplot
import pandas as pd
import numpy as np
np.random.seed(3)
```

```
# (x_train_image, y_trai
# upload a file to Googl

from google.colab import
uploaded = files.upload(
```

選擇檔案 未選擇任何檔案

開啟

USER › .keras › datasets ›

搜尋 datasets

組合管理 新增資料夾

名稱	修改日期	類型	大小
cifar-10-batches-py	2009/6/5 上午 04	檔案資料夾	
cifar-10-batches-py.tar.gz	2018/9/25 上午 1...	WinRAR 壓縮檔	166,503 KB
mnist.npz	2018/10/11 下午 ...	NPZ 檔案	11,222 KB

快速存取
下載
桌面
文件
圖片
本機
20181106
Output
Record
video
OneDrive
本機
3D 物件
下載

檔案名稱(N): mnist.npz

所有檔案

開啟(O) 取消

```
[34] # (x_train_image, y_train_label), (x_test_image, y_test_label) = mni
     # upload a file to Google CoLab

     from google.colab import files
     uploaded = files.upload() # then browse, select the files. It's ther
```

選擇檔案 mnist.npz

- **mnist.npz**(n/a) - 11490434 bytes, last modified: 2018/10/11 - 100% done

```
Saving mnist.npz to mnist.npz
```

```
# present working directory
!pwd
!mkdir sample_data
```

```
/content
mkdir: cannot create directory ‘sample_data’: File exists
```

```
[47] # list files in the directory
     !ls -l
```

```
total 11228
-rw-r--r-- 1 root root 11490434 Nov 11 08:19 mnist.npz
drwxr-xr-x 1 root root     4096 Nov 11 08:28 sample_data
```

```
[48] # copy a file to a directory
     !cp mnist.npz sample_data
```

```
[49] # list files in CoLab
     !ls -l
```

```
total 11228
-rw-r--r-- 1 root root 11490434 Nov 11 08:19 mnist.npz
drwxr-xr-x 1 root root     4096 Nov 11 08:28 sample_data
```

```
[50] # list files in CoLab
     !ls -l sample_data
```

```
total 11224
-rw-r--r-- 1 root root 11490434 Nov 11 08:30 mnist.npz
```

```
[52] # read in the file
     from numpy import load

     data = load('sample_data/mnist.npz')
     lst = data.files
     print(lst)
```

```
['x_test', 'x_train', 'y_train', 'y_test']
```

```
[53] x_test_image  = data['x_test']
     x_train_image = data['x_train']
     y_test_label  = data['y_test']
     y_train_label = data['y_train']
```

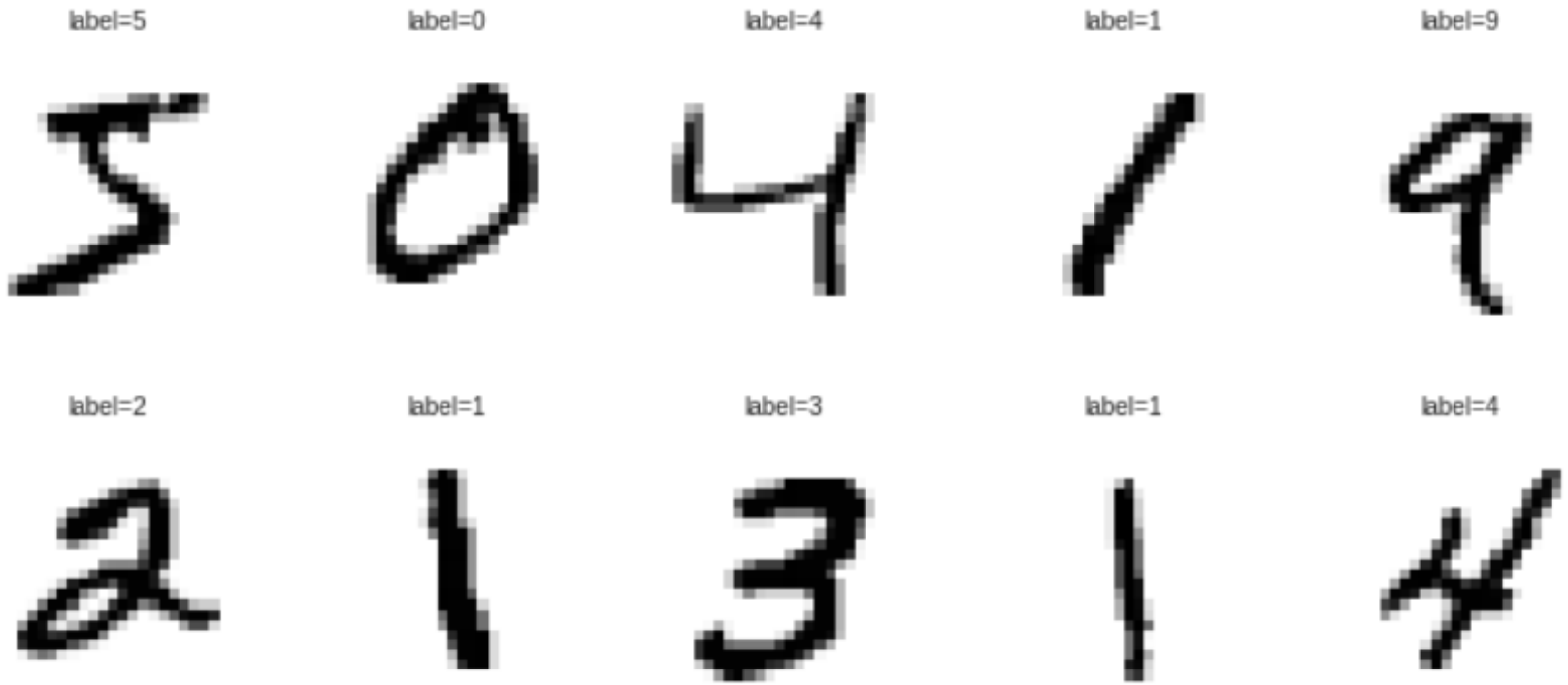
```
['x_test', 'x_train', 'y_train', 'y_test']
```

```
[53] x_test_image  = data['x_test']
     x_train_image = data['x_train']
     y_test_label  = data['y_test']
     y_train_label = data['y_train']

     print(x_train_image.shape)
     print(y_train_label.shape)
     print(x_test_image.shape)
     print(y_test_label.shape)
```

```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
```

(2) View the first 10 images and labels

```
[54] fig = plt.gcf()
     fig.set_size_inches(12,14)

     for i in range(0,10):
         ax=plt.subplot(5,5,1+i)
         ax.imshow(x_train_image[i], cmap='binary')
         title= "label=" +str(y_train_label[i])
         ax.set_title(title,fontsize=10)
         ax.set_xticks([]);ax.set_yticks([])
     plt.show()
```



(3) Convert 2-D image to nx28x28x1 array, normalize the numbers

```
[55] # convert 2-D 28x28 image to 4-D nx28x28x1  array

     x_Train4D=x_train_image.reshape(x_train_image.shape[0],28,28,1).astype('float32')
     x_Test4D=x_test_image.reshape(x_test_image.shape[0],28,28,1).astype('float32')
```

```
[56] # normalize the image numbers to 0~1

     x_Train4D_normalize = x_Train4D / 255
     x_Test4D_normalize = x_Test4D / 255
     print(x_Train4D_normalize.shape)
     print(x_Test4D_normalize.shape)
```

```
(60000, 28, 28, 1)
(10000, 28, 28, 1)
```

(4) Convert label number to one-hot encoding

```
[57] # convert label numbers to one-hot encoding

     y_TrainOneHot = np_utils.to_categorical(y_train_label)
     y_TestOneHot = np_utils.to_categorical(y_test_label)
     print(y_TrainOneHot.shape)
     print(y_TestOneHot.shape)
```

```
(60000, 10)
(10000, 10)
```

(5) Use a Convolutional Neural Network


```
[58] from keras.models import Sequential
     from keras.layers import Dense,Dropout,Flatten,Conv2D,MaxPooling2D
```

```
[59] model = Sequential()
```

```
[60] model.add(Conv2D(filters=16,
                      kernel_size=(5,5),
                      padding='same',
                      input_shape=(28,28,1),
                      activation='relu'))
```

```
[61] # Enable this cell in the second step

     model.add(MaxPooling2D(pool_size=(2, 2)))
     model.add(Conv2D(filters=36,
                      kernel_size=(5,5),
                      padding='same',
                      activation='relu'))
     model.add(MaxPooling2D(pool_size=(2, 2)))
     model.add(Dropout(0.25))
```

```
[62] model.add(Flatten())
```

```
[63] # Enable this cell in the second step

     model.add(Dense(128, activation='relu'))
     model.add(Dropout(0.5))
```

```
[64] model.add(Dense(10,activation='softmax'))
```

```
[65] print(model.summary())
```

```
[65] print(model.summary())
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 28, 28, 16)        416
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 14, 14, 16)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 14, 14, 36)        14436
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 7, 7, 36)          0
_________________________________________________________________
dropout_1 (Dropout)          (None, 7, 7, 36)          0
_________________________________________________________________
flatten_1 (Flatten)          (None, 1764)              0
_________________________________________________________________
dense_1 (Dense)              (None, 128)               225920
_________________________________________________________________
dropout_2 (Dropout)          (None, 128)               0
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1290
=================================================================
Total params: 242,062
Trainable params: 242,062
Non-trainable params: 0
_________________________________________________________________
None
```

(6) Model training

```
[66] model.compile(loss='categorical_crossentropy',
                   optimizer='adam',metrics=['accuracy'])
```

```
[67] train_history=model.fit(x=x_Train4D_normalize,
                             y=y_TrainOneHot,validation_split=0.2,
                             epochs=50, batch_size=300,verbose=2)
```

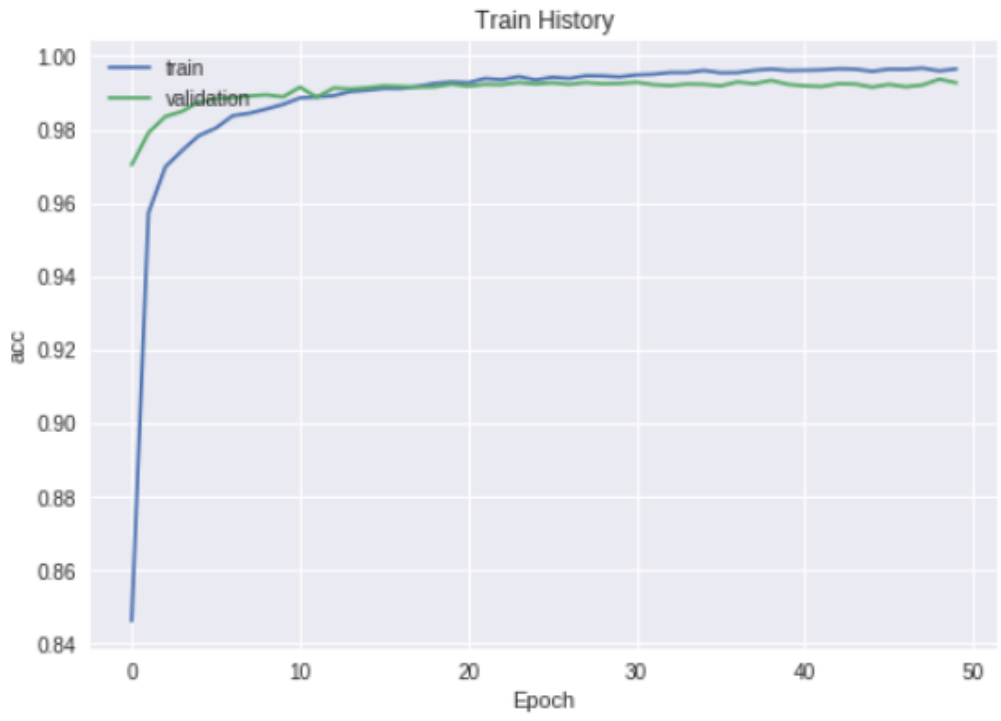
```
Train on 48000 samples, validate on 12000 samples
Epoch 1/50
 - 7s - loss: 0.4862 - acc: 0.8460 - val_loss: 0.1012 - val_acc: 0.9700
Epoch 2/50
 - 3s - loss: 0.1417 - acc: 0.9577 - val_loss: 0.0652 - val_acc: 0.9793
Epoch 3/50
 - 3s - loss: 0.1022 - acc: 0.9695 - val_loss: 0.0560 - val_acc: 0.9830
Epoch 4/50
 - 3s - loss: 0.0859 - acc: 0.9743 - val_loss: 0.0486 - val_acc: 0.9848
```

```
Epoch 40/50
 - 3s - loss: 0.0116 - acc: 0.9960 - val_loss: 0.0325 - val_acc: 0.9923
Epoch 41/50
 - 3s - loss: 0.0115 - acc: 0.9961 - val_loss: 0.0318 - val_acc: 0.9918
Epoch 42/50
 - 3s - loss: 0.0105 - acc: 0.9962 - val_loss: 0.0351 - val_acc: 0.9917
Epoch 43/50
 - 3s - loss: 0.0104 - acc: 0.9965 - val_loss: 0.0328 - val_acc: 0.9924
Epoch 44/50
 - 3s - loss: 0.0110 - acc: 0.9964 - val_loss: 0.0315 - val_acc: 0.9923
Epoch 45/50
 - 3s - loss: 0.0115 - acc: 0.9958 - val_loss: 0.0320 - val_acc: 0.9915
Epoch 46/50
 - 3s - loss: 0.0110 - acc: 0.9964 - val_loss: 0.0313 - val_acc: 0.9923
Epoch 47/50
 - 3s - loss: 0.0105 - acc: 0.9964 - val_loss: 0.0344 - val_acc: 0.9916
Epoch 48/50
 - 3s - loss: 0.0098 - acc: 0.9967 - val_loss: 0.0318 - val_acc: 0.9921
Epoch 49/50
 - 3s - loss: 0.0110 - acc: 0.9959 - val_loss: 0.0293 - val_acc: 0.9937
Epoch 50/50
 - 3s - loss: 0.0102 - acc: 0.9965 - val_loss: 0.0330 - val_acc: 0.9927
```

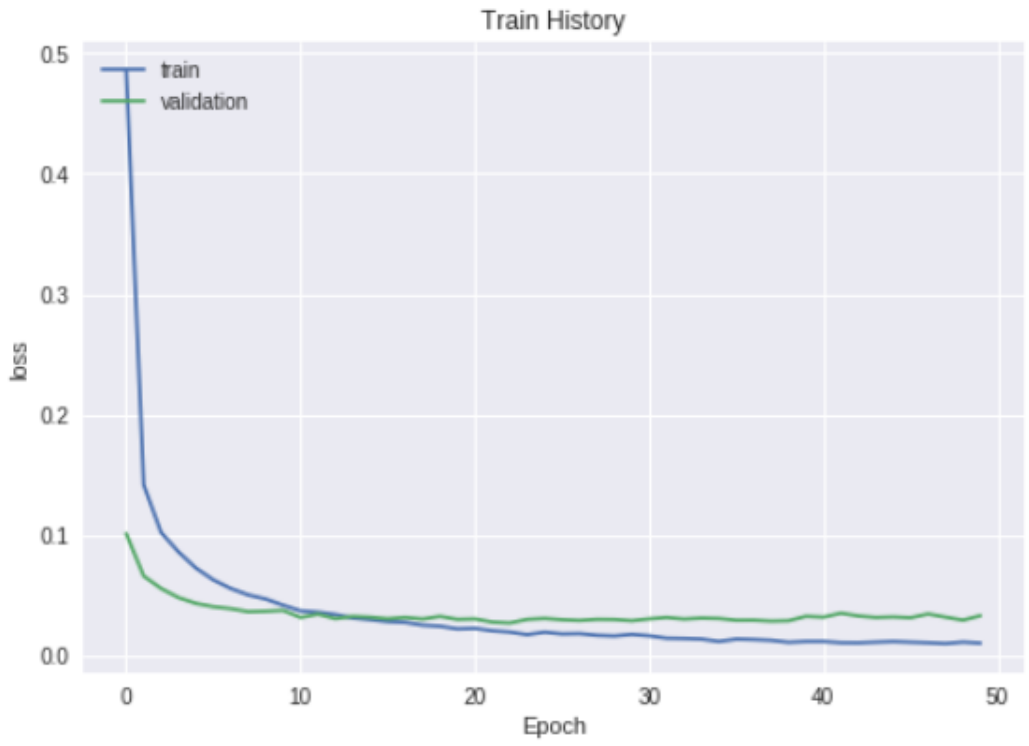
(7) Training history

```
[168] def show_train_history(train_history,train,validation):
          plt.plot(train_history.history[train])
          plt.plot(train_history.history[validation])
          plt.title('Train History')
          plt.ylabel(train)
          plt.xlabel('Epoch')
          plt.legend(['train', 'validation'], loc='upper left')
          plt.show()
```

```
[169] show_train_history(train_history,'acc','val_acc')
```



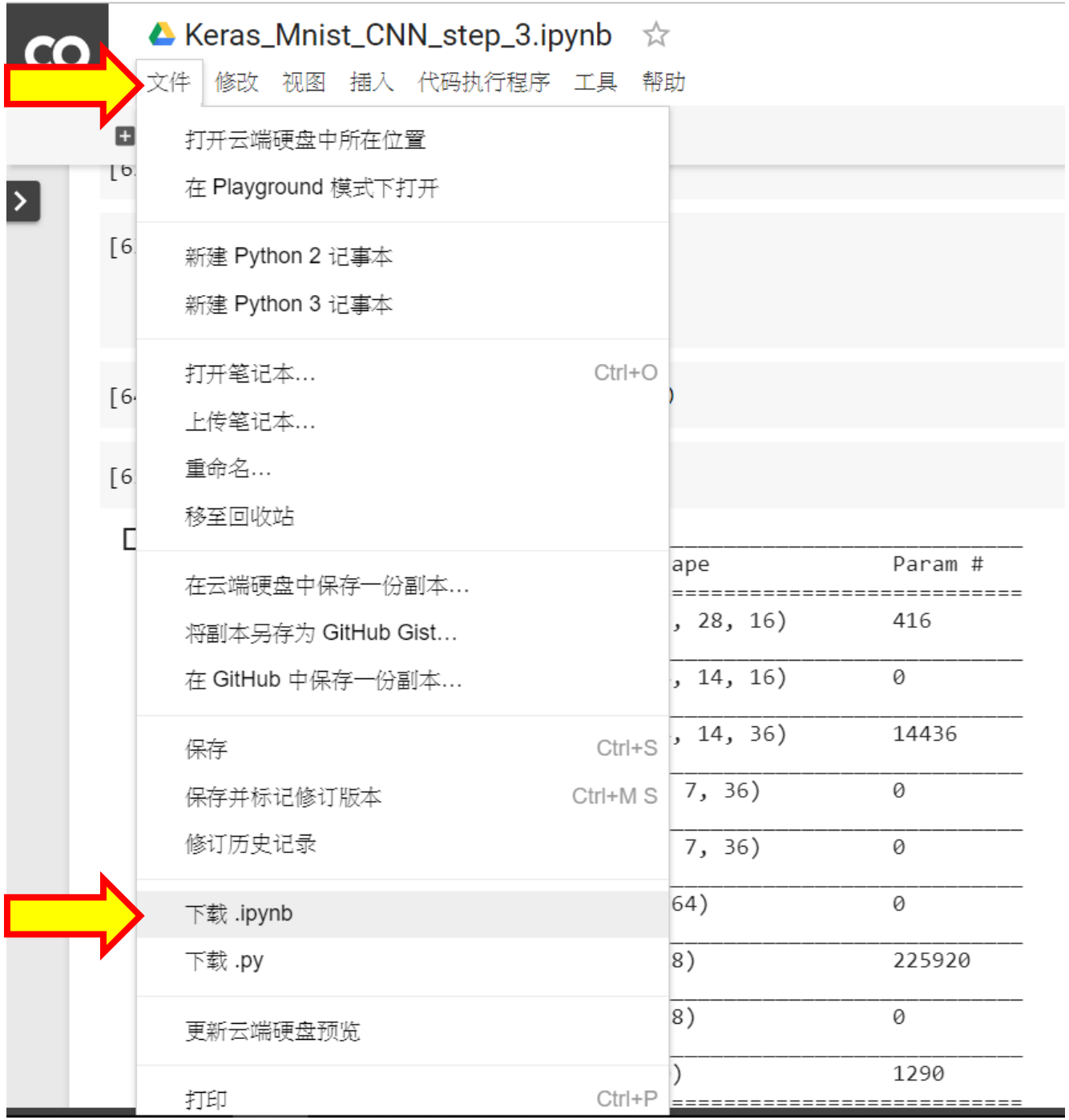

```
[170] show_train_history(train_history,'loss','val_loss')
```



(10) Confusion matrix

```
[175] pd.crosstab(y_test_label,prediction,
                  rownames=['label'],colnames=['predict'])
```

predict	0	1	2	3	4	5	6	7	8	9
label										
0	976	0	0	0	0	0	1	1	2	0
1	0	1131	1	2	0	0	0	0	1	0
2	0	1	1026	0	0	0	0	4	1	0
3	0	0	1	1005	0	2	0	0	2	0
4	0	0	0	0	977	0	1	0	1	3
5	1	0	0	6	0	883	1	0	1	0
6	3	1	0	0	2	2	948	0	2	0
7	0	1	5	0	0	0	0	1022	0	0
8	1	0	1	2	0	0	0	0	969	1
9	0	2	0	0	6	1	0	3	1	996


Keras_Mnist_CNN_step_3.ipynb
文件 修改 视图 插入 代码执行程序 工具 帮助
打开云端硬盘中所在位置
在 Playground 模式下打开
新建 Python 2 记事本
新建 Python 3 记事本
打开笔记本... Ctrl+O
上传笔记本...
重命名...
移至回收站
在云端硬盘中保存一份副本...
将副本另存为 GitHub Gist...
在 GitHub 中保存一份副本...
保存 Ctrl+S
保存并标记修订版本 Ctrl+M S
修订历史记录
下载 .ipynb
下载 .py
更新云端硬盘预览
打印 Ctrl+P
ape Param #
, 28, 16) 416
, 14, 16) 0
, 14, 36) 14436
7, 36) 0
7, 36) 0
64) 0
8) 225920
8) 0
) 1290

Homework:

(回家看) ML Lecture 10: Convolutional Neural Network

https://www.youtube.com/watch?v=FrKWiRv254g

請點選以下連結, 並親自體驗 CNN

https://cs.stanford.edu/people/karpathy/convnetjs/

Classify **MNIST** digits with a Convolutional Neural Network

Classify **CIFAR-10** with Convolutional Neural Network

Thank you for your attention